SOULHUNTER

Documentação Técnica • Sprint 4 — Domain Driven Design Using Java

NOME DA EQUIPE

SoulHunter

NOME DA SOLUÇÃO

SoulHunter — Sistema de Gamificação de Ações Sustentáveis

INTEGRANTES

Tárik Moussa Alma — RM: 571411

Giovanni Azevedo — RM: 572894

Fabricio Aquiles Sales da Silva — RM: 570985

Carlos Eduardo Tsucamoto Chiarelli — RM: 569574

Ítalo Neto — RM: 572912

Sumário

1. Objetivo e Escopo do Projeto 3

2. Descrição das Funcionalidades 4

3. Tabela de Endpoints (API Restful) 5

4. Protótipo do Sistema 7

- 4.1 Tela — Lista de Usuários (sem registros) 7
- 4.2 Tela — Cadastro de Usuário 7
- 4.3 Tela — Lista de Usuários (com registro) 8
- 4.4 Tela — Painel do Caçador: registrar reciclagem 8
- 4.5 Tela — Resultado da reciclagem 9
- 4.6 Tela — Painel do Caçador: registrar transporte 9
- 4.7 Tela — Resultado do transporte 10
- 4.8 Tela — Painel do Caçador: fantasma capturado 10
- 4.9 Tela — Lista de Fantasmas 11
- 4.10 Tela — Exclusão bloqueada pela regra de negócio 11
- 4.11 Tela — Validação de e-mail duplicado 11

5. Modelo de Entidade-Relacionamento (MER) 12

6. Diagrama de Classes 13

- 6.1 Camada Model (pacote to) 13
- 6.2 Estrutura relacionada à tabela Fantasma 14
- 6.3 Estrutura relacionada à tabela Usuário 15

7. Procedimentos para Rodar a Aplicação 16

- 7.1 Ferramentas Utilizadas 16
- 7.2 Como Importar e Configurar o Projeto 16
- 7.3 Passo a Passo da Execução da API 17
- 7.4 Executando o Front-end 17
- 7.5 Deploy no Render 17

1. Objetivo e Escopo do Projeto

O **SoulHunter** é um sistema de gamificação de ações sustentáveis. O propósito principal do sistema é incentivar usuários a adotarem hábitos ecologicamente responsáveis — como reciclagem de materiais e uso de transporte alternativo — recompensando essas ações com um recurso chamado **Essência**.

A cada ação sustentável registrada, o usuário acumula Essência e evolui seu **Título de Caçador** (Aprendiz, Iniciante, Raro ou Lendário), de acordo com a quantidade acumulada. Além do cadastro de usuários, o sistema permite capturar **Fantasmas** — elementos de gamificação com nome, tipo, raridade e valor em Essência — sempre vinculados a um usuário responsável.

Esta Sprint 4 é a **entrega final** do projeto. A aplicação, que na Sprint 3 era uma aplicação Java com interface por caixas de diálogo (`JOptionPane`), evoluiu para uma **API Restful** desenvolvida com o framework **Spring Boot**, organizada em camadas (`to` , `dao` , `bo` e `resource`) sobre o banco de dados **Oracle**, e integrada a um **front-end em React** que consome a API por requisições HTTP. A API está publicada em nuvem no **Render**.

O escopo desta etapa contempla o CRUD completo de Usuários e Fantasmas, as regras de negócio de Essência e Título de Caçador, a validação dos dados de entrada e o tratamento de erros com códigos de status HTTP. O sistema não contempla, nesta etapa, autenticação (login) de usuários nem operações de troca ou venda de Fantasmas entre usuários.

Links da entrega

Item	Endereço
API publicada (Render)	`https://soulhunter-rm571411.onrender.com`
Exemplo de consulta	`https://soulhunter-rm571411.onrender.com/soulhunter/usuarios`
Repositório (GitHub)	`https://github.com/cearaa/Sprint4JavaBDFront`

No plano gratuito do Render a API é suspensa após um período sem uso; a primeira requisição pode levar cerca de 1 minuto para ser respondida.

2. Descrição das Funcionalidades

- **Cadastrar Usuário** — insere um novo usuário (nome e e-mail) no banco de dados, com Essência inicial zerada e Título "Aprendiz de Caçador". O e-mail deve ser único.
- **Alterar Usuário** — atualiza nome e e-mail de um usuário existente. A Essência e o Título não podem ser editados diretamente, apenas através das ações sustentáveis.
- **Excluir Usuário** — remove um usuário do banco. Não é possível excluir um usuário que ainda possua Fantasmas vinculados.
- **Listar / Consultar Usuário** — lista todos os usuários ou consulta um usuário pelo código, incluindo a Essência e o Título de Caçador atuais.
- **Registrar Reciclagem** — soma Essência ao usuário com base na quantidade reciclada (5 pontos de Essência por quilo), recalculando o Título automaticamente.
- **Registrar Transporte** — soma Essência ao usuário com base na distância percorrida (3 pontos por km de bicicleta, 2 pontos por km em outros meios), recalculando o Título automaticamente.
- **Capturar Fantasma** — insere um novo Fantasma no banco, vinculado a um usuário existente.
- **Alterar Fantasma** — atualiza nome, tipo, raridade e valor de Essência de um Fantasma existente. O dono do Fantasma é mantido.
- **Excluir Fantasma** — remove um Fantasma do banco de dados.
- **Listar / Consultar Fantasma** — lista todos os Fantasmas, consulta um Fantasma pelo código ou lista os Fantasmas de um usuário.

Título de Caçador

O Título é recalculado a cada ação sustentável, de acordo com a Essência acumulada:

Essência acumulada	Título
menos de 30	Aprendiz de Caçador
de 30 até menos de 60	Caçador Iniciante
de 60 até menos de 100	Caçador Raro
100 ou mais	Caçador Lendário

Recursos técnicos da Sprint 4

- **Validações** — os dados recebidos são validados com Bean Validation (campos obrigatórios, e-mail válido, valores não negativos) e pelas regras da camada BO; dados inválidos resultam em resposta `400 Bad Request`.
- **Tratamento de erros** — todos os endpoints tratam exceções e respondem com o status HTTP adequado (200, 201, 204, 400, 404 ou 500) e uma mensagem clara.
- **CORS** — a classe `CorsFilter` libera o acesso do front-end à API.
- **Credenciais protegidas** — usuário e senha do banco não ficam no código: são lidos de variáveis de ambiente pela classe `ConnectionFactory`.
- **Deploy** — a API é publicada em nuvem (Render) por meio de um `Dockerfile`.

3. Tabela de Endpoints (API Restful)

A API segue os princípios REST: cada recurso possui uma URI, o verbo HTTP indica a operação (GET consulta, POST cria, PUT altera, DELETE exclui) e o código de status da resposta informa o resultado. A URL base em produção é `https://soulhunter-rm571411.onrender.com` e, na execução local, `http://localhost:8080`.

3.1 Recurso Usuários

Verbo	URI	Descrição	Status esperados
GET	`/soulhunter/usuarios`	Lista todos os usuários.	200, 500
GET	`/soulhunter/usuarios/{codigo}`	Busca um usuário pelo código.	200, 404, 500
GET	`/soulhunter/usuarios/{codigo}/fantasmas`	Lista os fantasmas capturados por um usuário.	200, 404, 500
POST	`/soulhunter/usuarios`	Cadastra um usuário (Essência inicia em 0).	201, 400, 500
PUT	`/soulhunter/usuarios/{codigo}`	Altera nome e e-mail do usuário.	201, 400, 404, 500
DELETE	`/soulhunter/usuarios/{codigo}`	Exclui o usuário (bloqueado se houver fantasmas).	204, 400, 404, 500
POST	`/soulhunter/usuarios/{codigo}/reciclagem`	Registra uma reciclagem e soma a Essência (5 por kg).	200, 400, 404, 500
POST	`/soulhunter/usuarios/{codigo}/transporte`	Registra um transporte sustentável e soma a Essência (bicicleta 3 por km; outros 2 por km).	200, 400, 404, 500

3.2 Recurso Fantasmas

Verbo	URI	Descrição	Status esperados
GET	`/soulhunter/fantasmas`	Lista todos os fantasmas.	200, 500
GET	`/soulhunter/fantasmas/{codigo}`	Busca um fantasma pelo código.	200, 404, 500
POST	`/soulhunter/fantasmas`	Captura (cadastra) um fantasma para um usuário existente.	201, 400, 500
PUT	`/soulhunter/fantasmas/{codigo}`	Altera nome, tipo, raridade e valor de Essência (o dono é mantido).	201, 400, 404, 500
DELETE	`/soulhunter/fantasmas/{codigo}`	Exclui o fantasma.	204, 404, 500

O verbo PUT responde `201 Created` em caso de sucesso, conforme o padrão adotado em aula.

3.3 Códigos de status

Código	Significado	Quando ocorre
200	OK	Consulta realizada ou ação (reciclagem/transporte) registrada com sucesso.
201	Created	Registro cadastrado (POST) ou alterado (PUT) com sucesso.
204	No Content	Registro excluído com sucesso (DELETE).
400	Bad Request	Dados inválidos ou regra de negócio violada (por exemplo: e-mail já cadastrado, quilos menores ou iguais a zero, exclusão de usuário que possui fantasmas).
404	Not Found	O usuário ou o fantasma informado na URI não existe.
500	Internal Server Error	Falha interna, como erro de comunicação com o banco de dados.

3.4 Exemplos de corpo das requisições (JSON)

```
POST /soulhunter/usuarios
{ "nome": "Maria Silva", "email": "maria@email.com" }

POST /soulhunter/usuarios/1/reciclagem
{ "tipoMaterial": "Plástico", "quilos": 2 }

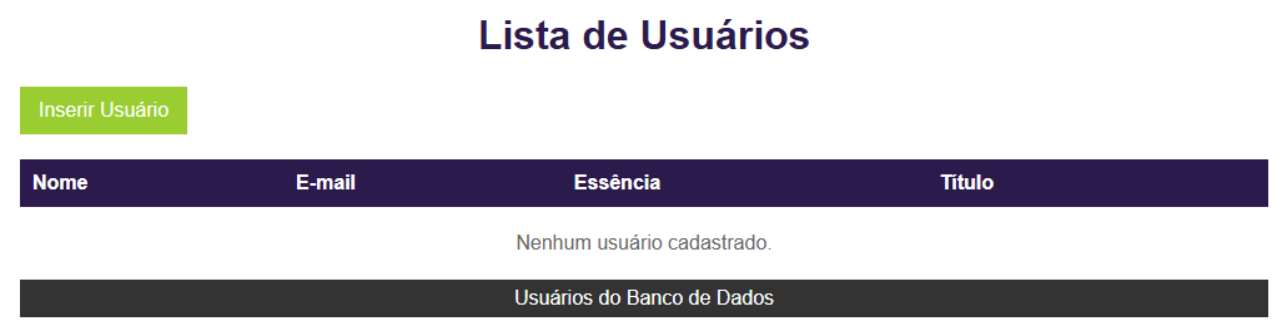
POST /soulhunter/usuarios/1/transporte
{ "tipoTransporte": "Bicicleta", "km": 10 }

POST /soulhunter/fantasmas
{ "nome": "Espectro Verde", "tipo": "Natureza", "raridade": "Raro", "valorEssencia": 15, "codUsuario": 1 }
```

4. Protótipo do Sistema

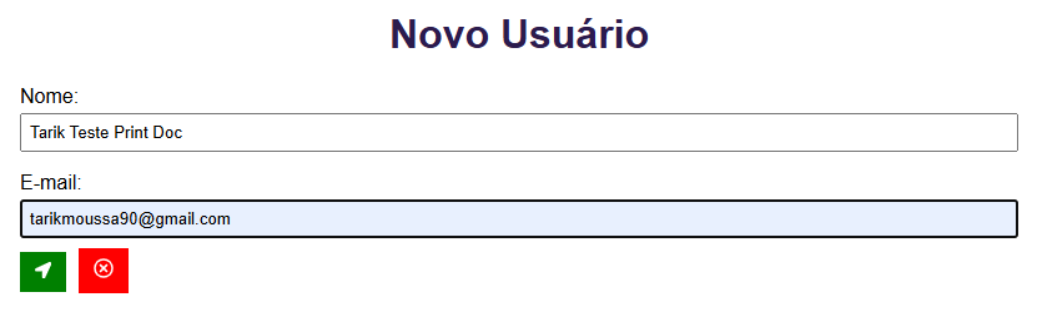
O front-end do SoulHunter foi desenvolvido em **React** e consome a API Restful publicada no Render. As telas a seguir são capturas reais do sistema em funcionamento (executado em `http://localhost:3000`), apresentadas na ordem de uso: cadastro de um usuário, registro de ações sustentáveis, captura de fantasmas e as validações da API.

4.1 Tela — Lista de Usuários (sem registros)



Interação: tela inicial do sistema. Exibe os usuários em uma tabela (nome, e-mail, Essência e Título); quando não há registros, mostra "Nenhum usuário cadastrado.". O botão **Inserir Usuário** abre o formulário de cadastro e o menu superior alterna entre **Usuários** e **Fantasmas**.
Chamada à API: `GET /soulhunter/usuarios`

4.2 Tela — Cadastro de Usuário



Interação: formulário com nome e e-mail (ambos obrigatórios). O botão verde envia os dados e o botão vermelho cancela e volta à lista. A Essência e o Título não são digitados: a API define Essência 0 e Título "Aprendiz de Caçador" para todo usuário novo.
Chamada à API: `POST /soulhunter/usuarios`

4.3 Tela — Lista de Usuários (com registro)



Interação: após salvar, o usuário aparece na lista com Essência 0,00 e Título "Aprendiz de Caçador". Cada linha tem três ícones: **olho** (abre o Painel do Caçador), **lápis** (edita nome e e-mail) e **lixeira** (exclui, pedindo confirmação).
Chamada à API: `GET /soulhunter/usuarios` (editar: `PUT /soulhunter/usuarios/{codigo}` ; excluir: `DELETE /soulhunter/usuarios/{codigo}`)

4.4 Tela — Painel do Caçador: registrar reciclagem



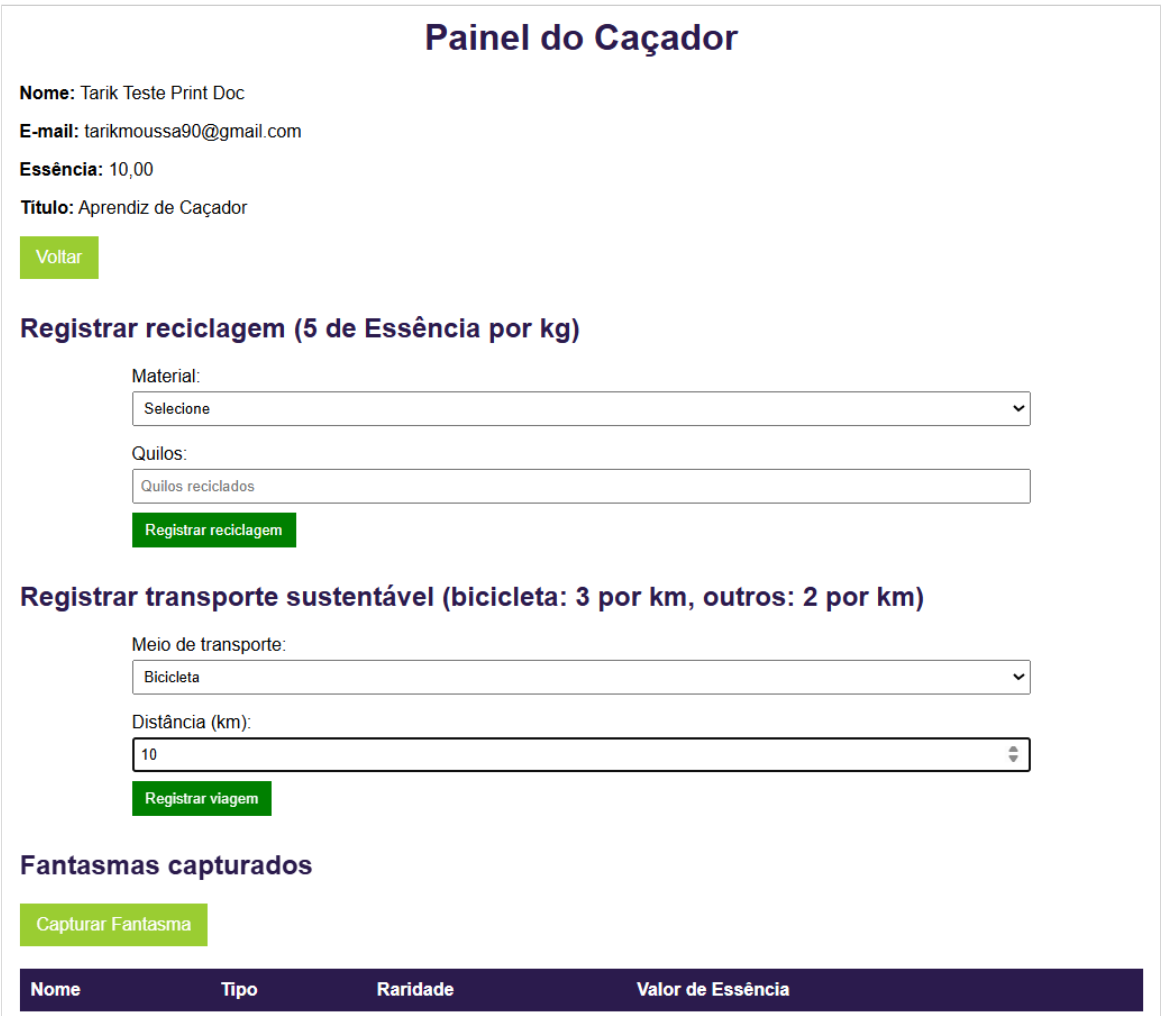
Interação: o painel reúne os dados do usuário e os formulários de ação. Para registrar uma reciclagem, o usuário escolhe o material (neste exemplo, Plástico) e informa os quilos (2). Ao clicar em **Registrar reciclagem**, a API calcula 5 pontos de Essência por quilo. O painel também mostra a lista de fantasmas capturados pelo usuário.
Chamada à API: `GET /soulhunter/usuarios/{codigo}` , `GET /soulhunter/usuarios/{codigo}/fantasmas` e `POST /soulhunter/usuarios/{codigo}/reciclagem`

4.5 Tela — Resultado da reciclagem



Interação: após registrar 2 kg, a Essência passou de 0,00 para 10,00 (2 × 5). O Título continua "Aprendiz de Caçador", pois a Essência ainda é menor que 30.
Chamada à API: `GET /soulhunter/usuarios`

4.6 Tela — Painel do Caçador: registrar transporte



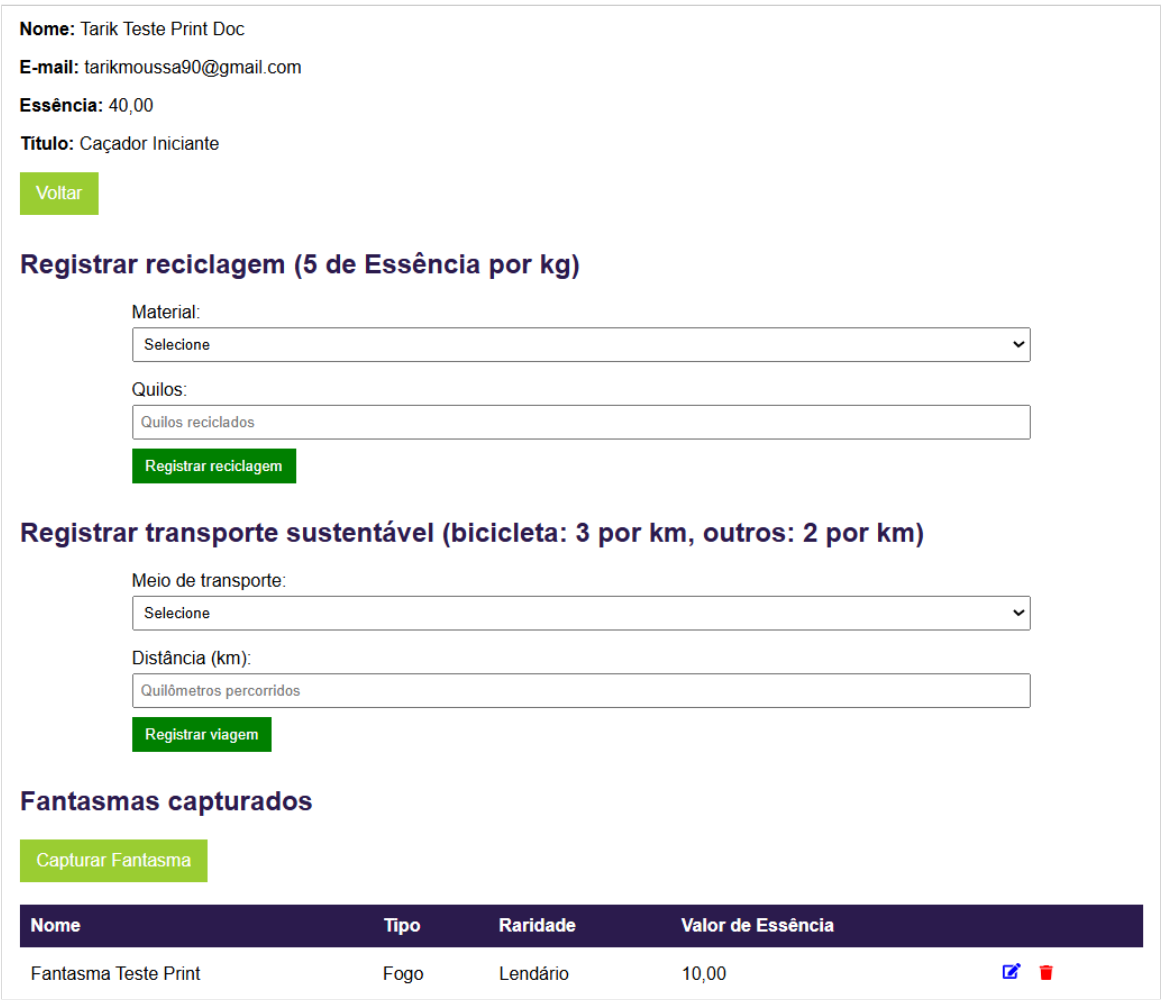
Interação: no segundo formulário, o usuário escolhe o meio de transporte (Bicicleta) e informa a distância (10 km). A API calcula 3 pontos de Essência por km para bicicleta (2 pontos por km nos demais meios). O painel mostra a Essência antes da ação (10,00).
Chamada à API: `POST /soulhunter/usuarios/{codigo}/transporte`

4.7 Tela — Resultado do transporte



Interação: a Essência passou de 10,00 para 40,00 (10 km × 3 = 30) e o Título foi recalculado automaticamente para "Caçador Iniciante" (Essência de 30 até menos de 60).
Chamada à API: `GET /soulhunter/usuarios`

4.8 Tela — Painel do Caçador: fantasma capturado



Interação: a tabela "Fantasmas capturados" lista os fantasmas do usuário (nome, tipo, raridade e valor de Essência), com ícones para editar e excluir. O botão **Capturar Fantasma** abre o formulário (nome, tipo, raridade e valor de Essência) e vincula o novo fantasma ao usuário do painel.
Chamada à API: `GET /soulhunter/usuarios/{codigo}/fantasmas`, `POST /soulhunter/fantasmas`, `PUT /soulhunter/fantasmas/{codigo}` e `DELETE /soulhunter/fantasmas/{codigo}`

4.9 Tela — Lista de Fantasmas



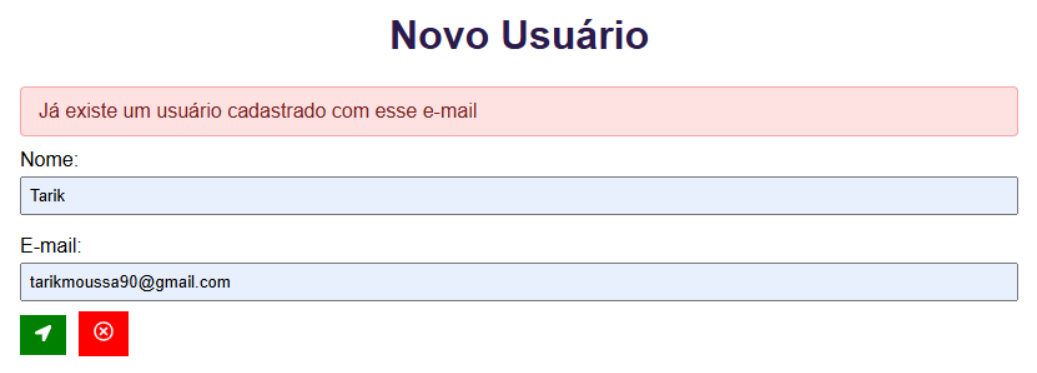
Interação: acessada pelo menu **Fantasmas**, lista todos os fantasmas do banco com o nome do caçador dono (link para o painel dele), além das ações de editar e excluir.
Chamada à API: `GET /soulhunter/fantasmas` e `GET /soulhunter/usuarios` (para exibir o nome do dono)

4.10 Tela — Exclusão bloqueada pela regra de negócio



Interação: ao tentar excluir um usuário que ainda possui fantasmas, a API recusa a operação (`400 Bad Request`) e o front-end exibe a mensagem da regra de negócio em destaque. O usuário permanece na lista; para excluí-lo, é preciso antes excluir seus fantasmas.
Chamada à API: `DELETE /soulhunter/usuarios/{codigo}` (resposta 400)

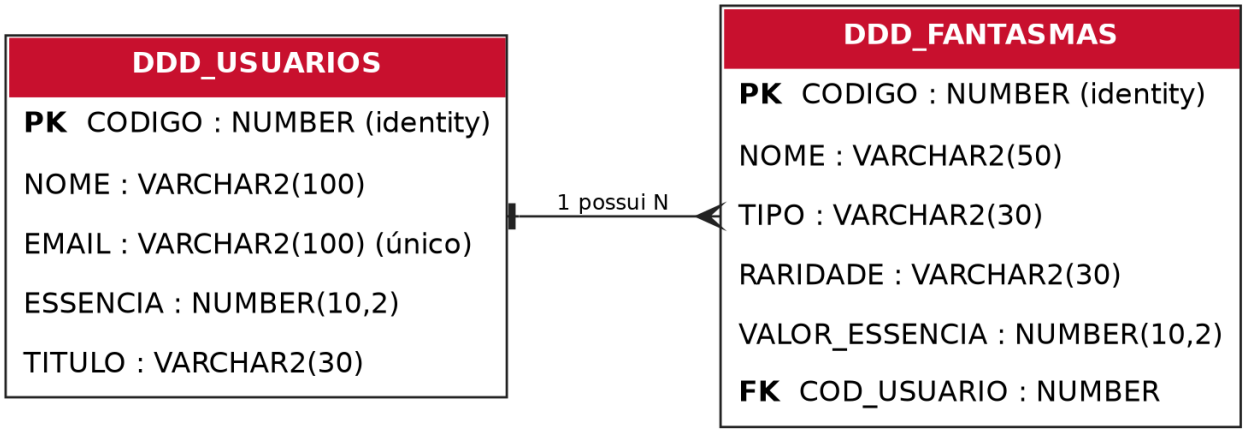
4.11 Tela — Validação de e-mail duplicado



Interação: ao cadastrar um e-mail que já existe, a API responde `400 Bad Request` com a mensagem "Já existe um usuário cadastrado com esse e-mail", exibida acima do formulário, que mantém os dados digitados para correção.
Chamada à API: `POST /soulhunter/usuarios` (resposta 400)

5. Modelo de Entidade-Relacionamento (MER)

O banco de dados Oracle utilizado pelo sistema é composto por duas tabelas relacionadas: a tabela `DDD_USUARIOS`, responsável pelo cadastro e pela Essência/Título de cada usuário, e a tabela `DDD_FANTASMAS`, que representa os Fantasmas capturados, cada um vinculado a um único usuário através da chave estrangeira `COD_USUARIO`. As chaves primárias (`CODIGO`) são geradas automaticamente pelo banco (`GENERATED ALWAYS AS IDENTITY`) e o e-mail do usuário é único.



Um usuário pode possuir vários Fantasmas (relacionamento 1:N). O banco recusa a exclusão de um usuário enquanto ele ainda possuir Fantasmas vinculados, preservando a integridade referencial; a camada BO verifica essa condição antes e devolve uma mensagem clara ao front-end.

Correspondência com a camada Model

Tabela	Classe Model (pacote `to`)	Observação
`DDD_USUARIOS`	`UsuarioTO`	Atributos `codigo`, `nome`, `email`, `essencia` e `titulo`.
`DDD_FANTASMAS`	`FantasmaTO`	Atributos `codigo`, `nome`, `tipo`, `raridade`, `valorEssencia` e `codUsuario`.

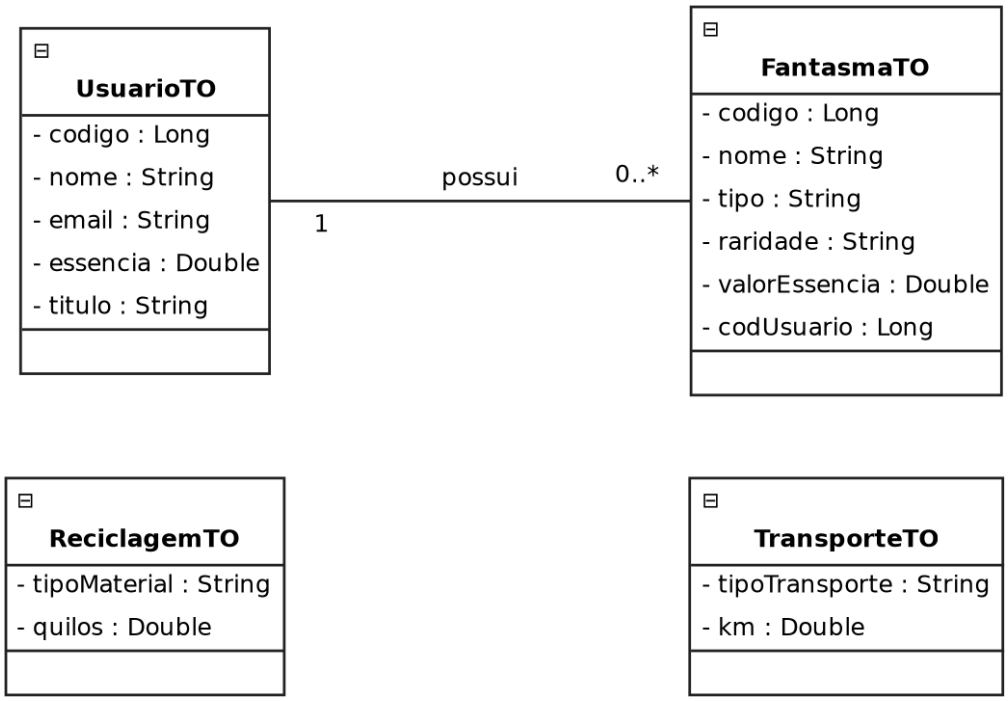
6. Diagrama de Classes

Os diagramas a seguir representam as classes efetivamente implementadas no projeto entregue, organizadas nas camadas `to` (Model), `dao`, `bo` e `resource`, seguindo o mesmo padrão visual apresentado em aula. As setas tracejadas indicam dependência (uma classe utiliza a outra) e a linha contínua indica a associação entre as classes. Por clareza, cada tabela do sistema é apresentada em uma página própria.

Pacote	Classes	Responsabilidade
`to`	UsuarioTO, FantasmaTO, ReciclagemTO, TransporteTO	Model: representa as entidades do banco e os corpos das requisições, com Bean Validation.
`dao`	ConnectionFactory, UsuarioDAO, FantasmaDAO	Acesso ao Oracle (JDBC com `PreparedStatement`) e CRUD.
`bo`	UsuarioBO, FantasmaBO	Regras de negócio e validações.
`resource`	UsuarioResource, FantasmaResource, CorsFilter	Endpoints REST e liberação de CORS.

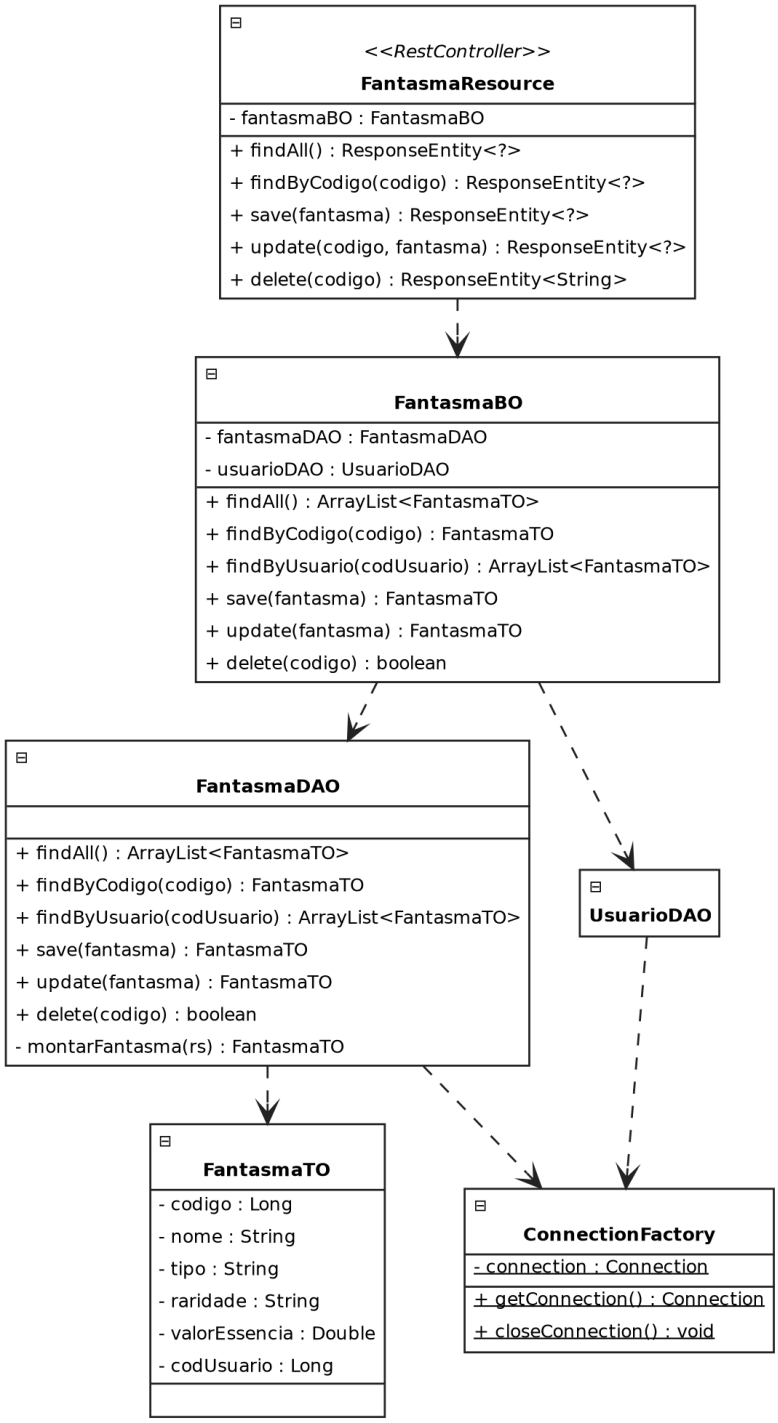
6.1 Camada Model (pacote to)

Um `UsuarioTO` se relaciona com zero ou mais `FantasmaTO` (atributo `codUsuario`, que corresponde à chave estrangeira do banco). `ReciclagemTO` e `TransporteTO` não são tabelas: carregam os dados enviados nas requisições de reciclagem e de transporte.



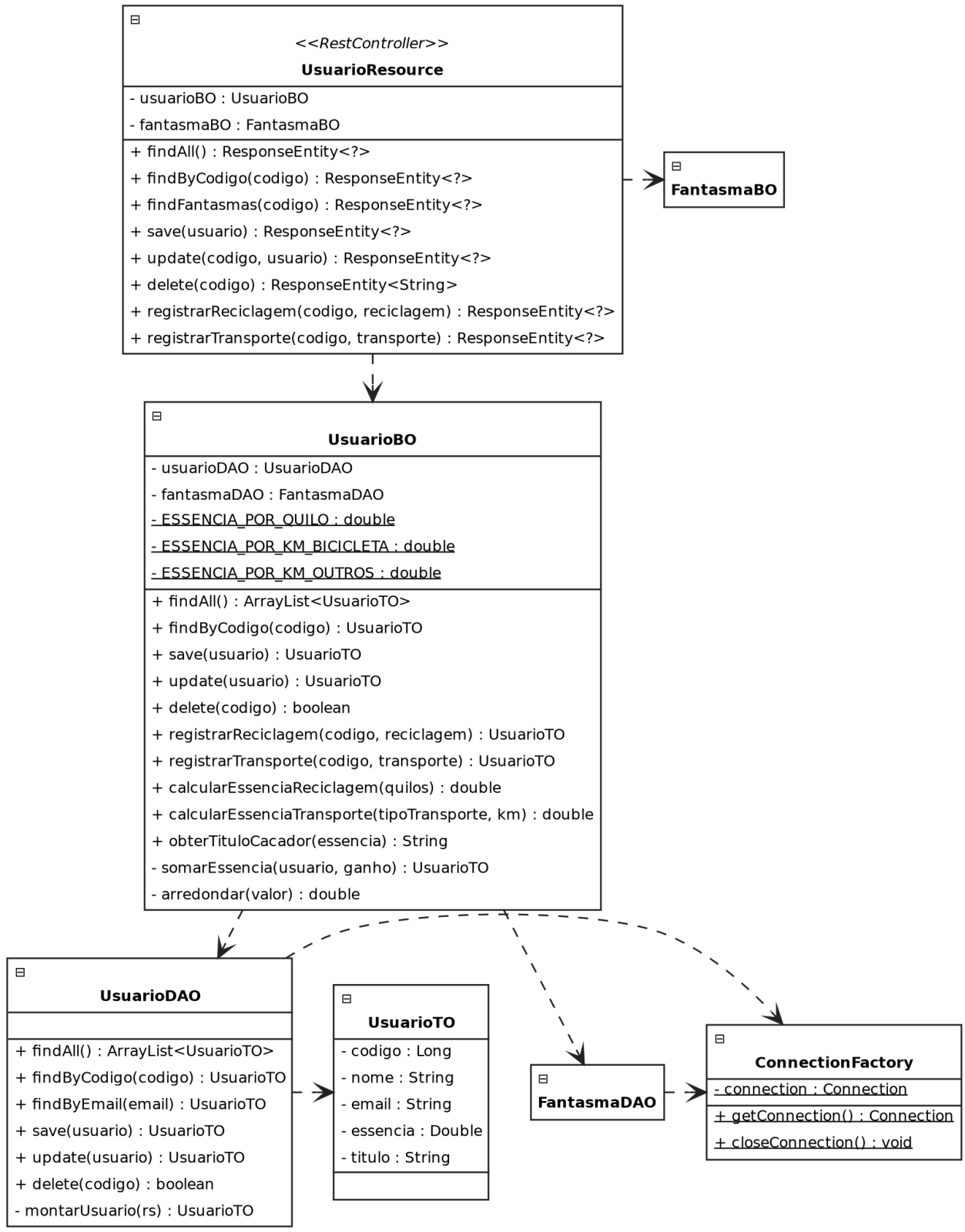
6.2 Estrutura relacionada à tabela Fantasma

`FantasmaBO` consulta `UsuarioDAO` para garantir que o dono do fantasma existe.



6.3 Estrutura relacionada à tabela Usuário

`UsuarioBO` consulta `FantasmaDAO` para impedir a exclusão de usuário com fantasmas e `UsuarioResource` usa `FantasmaBO` para listar os fantasmas de um usuário.



7. Procedimentos para Rodar a Aplicação

7.1 Ferramentas Utilizadas

Ferramenta	Detalhe
IDE	IntelliJ IDEA
JDK	JDK 21 ou superior (desenvolvido com Oracle GraalVM 25+37-LTS-jvmci-b01)
Framework	Spring Boot 3.5.7 (Spring Web e Bean Validation)
Build	Maven (Maven Wrapper `mvnw` incluído no projeto)
Banco de Dados	Oracle (servidor `oracle.fiap.com.br:1521:ORCL`), acessado pelo Oracle SQL Developer
Driver JDBC	`ojdbc11` (dependência Maven)
Front-end	React (Create React App), com Node.js 18 ou superior e npm
Publicação	GitHub e Render (Docker)

Variáveis de ambiente utilizadas pela API para conectar ao Oracle:

DB_URL	`jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL`
DB_USER	`rm571411`
DB_PASSWORD	`030208`

O usuário e a senha não ficam gravados no código: a classe `ConnectionFactory` os lê das variáveis de ambiente (no IntelliJ, em Run > Edit Configurations > Environment variables; no Render, em Environment Variables).

7.2 Como Importar e Configurar o Projeto

1. Obter o projeto: extrair o arquivo `.zip` em uma pasta local (ou clonar o repositório do GitHub).
2. Abrir o IntelliJ IDEA e selecionar `File > Open`, apontando para a pasta do projeto que contém o arquivo `pom.xml`.
3. Aguardar o Maven baixar as dependências (se necessário, clicar em `Sync Maven Changes`).
4. Confirmar que o SDK do projeto é um JDK 21 ou superior em `File > Project Structure > SDK`.

7.3 Passo a Passo da Execução da API

1. No Oracle SQL Developer, conectado com o schema do aluno, executar o script de criação das tabelas:

```
CREATE TABLE DDD_USUARIOS (
    CODIGO   NUMBER GENERATED ALWAYS AS IDENTITY,
    NOME     VARCHAR2(100) NOT NULL,
    EMAIL    VARCHAR2(100) NOT NULL,
    ESSENCIA NUMBER(10,2) DEFAULT 0 NOT NULL,
    TITULO   VARCHAR2(30),
    PRIMARY KEY (CODIGO),
    UNIQUE (EMAIL)
);

CREATE TABLE DDD_FANTASMAS (
    CODIGO         NUMBER GENERATED ALWAYS AS IDENTITY,
    NOME           VARCHAR2(50) NOT NULL,
    TIPO           VARCHAR2(30) NOT NULL,
    RARIDADE       VARCHAR2(30) NOT NULL,
    VALOR_ESSENCIA NUMBER(10,2) NOT NULL,
    COD_USUARIO    NUMBER NOT NULL,
    PRIMARY KEY (CODIGO),
    FOREIGN KEY (COD_USUARIO) REFERENCES DDD_USUARIOS (CODIGO)
);
```

2. No IntelliJ, abrir `Run > Edit Configurations`, selecionar `SoulHunterApplication` e preencher o campo `Environment variables` com os dados de conexão:

```
DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL;DB_USER=seu_usuario;DB_PASSWORD=sua_senha
```

3. Executar a classe `br.com.fiap.SoulHunterApplication` (botão direito > Run). Quando o console exibir `Started SoulHunterApplication`, a API estará disponível em `http://localhost:8080`.
4. Testar no navegador (ou no Insomnia) o endereço `http://localhost:8080/soulhunter/usuarios`: com o banco vazio, a resposta esperada é `[]` com status 200.
5. Opcional — gerar e executar o arquivo `.jar`: no terminal do IntelliJ, na pasta do projeto, executar `./mvnw clean -DskipTests package` e depois `java -jar target/soulhunter-0.0.1-SNAPSHOT.jar` (as variáveis de ambiente precisam estar definidas no terminal).

7.4 Executando o Front-end

1. Instalar o Node.js (versão 18 ou superior).
2. No terminal, dentro da pasta do front-end, executar `npm install`.
3. Conferir o endereço da API no arquivo `src/api.js` (constante `API_URL`), que já aponta para o Render. Para usar a API local, trocar por `http://localhost:8080/soulhunter`.
4. Executar `npm start` e abrir `http://localhost:3000` no navegador.

7.5 Deploy no Render

1. Enviar o projeto para um repositório no GitHub (`git init`, `git add .`, `git commit`, `git push`).
2. No Render, escolher `New > Web Service`, selecionar o repositório e preencher: **Name** `soulhunter-rm571411`, **Language** `Docker`, **Instance Type** `Free`.
3. Em `Environment Variables`, cadastrar `DB_URL`, `DB_USER` e `DB_PASSWORD`.
4. Clicar em `Deploy Web Service` e acompanhar os logs até a mensagem `Your service is live`.
5. Testar o endereço `https://soulhunter-rm571411.onrender.com/soulhunter/usuarios`: a resposta esperada, com o banco vazio, é `[]`.